

请参阅此出版物的讨论、统计数据和作者简介: <https://www.researchgate.net/publication/232621717>

## 挖掘 Git 的承诺和风险

文章 · 2009 年 5 月

DOI: 10.1109/MSR.2009.5069475 · 来源: DBLP

引文

282

读

1,555

6 位作者, 包括:



克里斯蒂安·伯德  
Microsoft

93 篇出版物 8,322 次引用

[查看资料](#)



厄尔 T. 巴尔  
伦敦大学学院

108 篇出版物 7,778 次引用

[查看资料](#)



大卫·汉密尔顿  
加州大学戴维斯分校

5 篇出版物 463 次引用

[查看资料](#)



普雷姆库马尔·德万布  
加州大学戴维斯分校

232 篇出版物 15,999 次引用

[查看资料](#)

# 挖掘 Git 的承诺和风险

Christian Bird\*, Peter C. Rigby†, Earl T. Barr\*, David J. Hamilton\*, Daniel M. German†, Prem Devanbu\*

\*美国加州大学戴维斯分校 †加拿大维多利亚大学

{鸟, 巴尔, 哈米尔托德, 德万布}@cs.ucdavis.edu {pcr, dmg}@cs.uvic.ca

## 抽象

我们现在见证了去中心化源代码管理（DSCM）系统的快速增长，其中每个开发人员都有自己的存储库。DSCM 促进了一种协作方式，在这种协作方式中，工作输出可以在协作者之间横向（和私下）流动，而不是总是通过中央存储库上下（和公开）流动。去中心化既带来了新数据的承诺，也带来了误解的危险。我们专注于 git，这是一种非常流行的 DSCM，用于备受瞩目的项目。去中心化和 git 的其他功能（例如自动记录的贡献者归属）导致了更丰富的内容历史记录，从而引发了新的问题，例如“贡献如何在开发人员之间流向官方项目存储库？但是，也存在陷阱。提交在存储库之间移动时，可以重新排序、删除或编辑。SCM 和 DSCM 通用术语的语义有时会明显不同，这可能会造成混淆。例如，在集中式 SCM 中，所有开发人员都可以立即看到提交，但在 DSCM 中则不可见。我们的目标是帮助对 DSCM 感兴趣的研究人员在挖掘和分析 git 数据时避免这些和其他风险。

从手上得分的茎中  
我在疲惫的土地上拧干它。

A. E. Housman, 什罗普郡小伙子

## 1. 引言

自世纪之交以来，研究人员一直在利用 SCM 存储库中的数据，这些数据已免费提供给开源软件（OSS）项目。这些数据已被用于重建创建软件的过程 [1], [2]。研究人员还使用这些数据创建了推荐系统 [3]、[4]、[5]、研究进化模式 [6]、[7]、[8]、预测错误 [9]、[10]、[11]，并检查协作 [12]、[13]、[14]。使用 DSCM 的软件项目数量有所增加，并且看起来将继续增加。图 1 显示

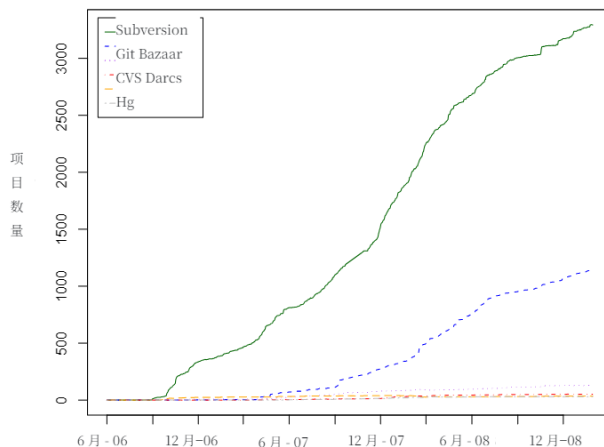


图 1. Debian 项目对 SCM 的使用。

在一段时间内报告使用给定 SCM 的具有 Debian 软件包的项目数量 1。截至 2009 年 2 月，36% 的软件包包含 SCM 信息。虽然不完整，但这些数据有力地表明 git 的使用量仅次于 SVN，并且它的使用量正在增长。事实上，git 也被许多备受瞩目的 OSS 项目采用，例如 X.org、Ruby on Rails、Wine、Samba、Perl 和 Glasgow Haskell 编译器。

这些项目和其他项目的存储库引起了研究人员的兴趣，但它们的数据与集中式项目中的数据在重要方面有所不同。

Massey 和 Packard [15] 提出了一种将 CSCM 转换为 git 以挖掘数据的方法。然而，据我们所知，只有一篇论文 [16] 研究了从基于 git 的项目中挖掘的数据。本文介绍了从 Linux git 存储库中提取的数据分析结果。我们还在 Linux Weekly News 中找到了一篇文章，该文章使用从 git 挖掘的数据来跟踪补丁如何从子系统 git 存储库进入稳定的主线 linux 树 [17]。本文和文章都没有讨论 git 和集中式 SCM 之间的核心区别。

1. 根据 2006 年引入 Debian 软件包描述的 vcs-（SCM-）头文件的项目提供的的数据。

这些差异导致了 SCM 和 DSCM 中截然不同的实践。在 DSCM 数据中可以观察到全新的现象，这导致了许多新问题。这些数据为研究人员提供了许多新的机会，但也充满了概念和实际风险。在挖掘和分析的过程中，由于对数据以及导致我们观察到的数据的过程的理解不完整，我们遭受了许多挫折并得出了错误的初始结论。

我们比较了 SVN 和 git，它们是集中式和分散式的流行和代表性 SCM。两者之间最根本的区别是，在 git 下，开发人员可以以托管方式共享代码，而无需使用主存储库，提交会影响所有其他开发人员。这是因为开发人员有自己的存储库，每个存储库都可以讲述软件项目故事的不同部分。

承诺 1：由于 git 项目上的任何开发人员都可以公开访问自己的存储库，因此可以恢复更多历史记录，包括正在进行的工作和从未进入稳定代码库的工作。

任何人都可以公开访问自己的 git 存储库，并且已经存在免费的 git 托管服务，例如 GitHub、Graphical 和 repo.or.cz。例如，在撰写本文时，有 481 个可公开访问的开发人员 git 存储库是从 Ruby on Rails2 克隆的。

不同的开发人员在其存储库中可以有不同的内容。例如，Ubuntu 维护自己的 Linux 内核存储库，其中包含自己的更改，其中一些更改从未进入官方内核树。Linux 开发人员 David Miller 维护着一个名为 net-next-2.6 的存储库，其中包含新代码或实验性代码，并且是内核中网络代码的暂存区域。

尽管采用去中心化模型，但许多 OSS 项目都有一个“官方”存储库，开发人员从该存储库进行发布，开发人员将其用作源代码。这些存储库以及更活跃的开发人员的存储库可能是研究人员最感兴趣的存储库。

在使用 SVN 的项目中，提交通常只有在经过审查后才会进入仓库；其余的活动（错误的开始、实验）在很大程度上是不可见的。使用 git，通过从开发人员的存储库中挖掘，可以恢复更完整的开发过程图，包括未进入稳定代码库的未打磨的实验性工作。

我们首先讨论了挖掘 git 的前景和风险，并阐明了集中式和分散式 SCM 世界之间的概念差异。

2. 根据 Github.com 2009 年 2 月的报告。

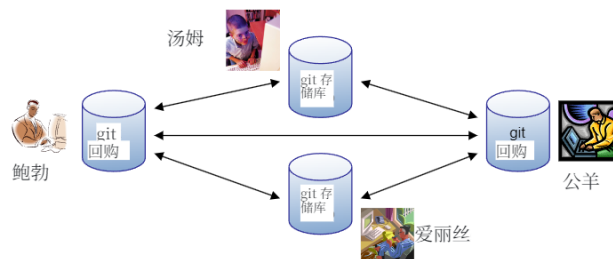


图 2.git 的分散式用法示例

## 概念差异

危险 1：Git 命名法与集中式 SCM（CSCM）不同：a）类似的作具有不同的命令；b）共享术语可以具有不同的含义。

图 2 说明了一个 4 人团队的 DSCM 世界，该团队由 Tom、Bob、Alice 和 Ram 组成，他们在一个大型项目进行协作。没有集中式存储库：每个开发人员都有自己的存储库，他们使用这些存储库进行提交、签出不同版本、比较版本差异等。

这看起来与集中式 SCM 世界大不相同，并产生了命名差异，即使是那些熟悉 git 的人也会感到困惑；尤其是那些语义略有不同的术语。为了引导我们的分析，我们在这里大致描述了这些差异，并在特定承诺和风险的背景下根据需要深入研究更细微的差异。

在 SVN 和 git 中，工作副本是存储库的当前签出状态。开发人员 Alice 执行 svn checkout 或 git clone 来创建一个工作副本。Alice 现在可以工作，并在准备好时使用 SVN 和 git 中的 commit 命令提交她的贡献。根本区别在于，在 SVN 中，提交被发送到中央仓库，但在 git 中，它们仍然是本地的。因此，SVN 提交对所有开发人员都是可见的；git 提交可能不是。在 SVN 中，Ram 必须发出 svn update 才能看到 Alice 刚刚提交的更改。在 git 下，仅当 a）另一个开发人员（例如 Ram）通过 git pull 将更改拉取到其存储库的当前分支或 b）Alice 使用 git push 将她的更改推送到远程存储库时，Alice 的提交才会移动到另一个存储库。

拉取 git 与 SVN 更新的不同之处在于，开发人员可以从任意数量的远程仓库拉取到自己的仓库中。对于 git 开发人员来说，拉取比推送更常见，因为推送需要写入权限，并且每个开发人员都“拥有”他的存储库并决定进入其中的内容。但是，git 确实通过裸存储库支持集中式工作流程。裸存储库不是

由单个人独家拥有，并且没有推送可能会中断的工作副本。

在 SVN 中，提交由序号标识，确定它们的顺序很简单。在 git 中，每个提交都由其内容的 SHA-1 哈希标识，并且仅给定两个 git 提交标识符，无法确定之前执行了哪个提交。

CSCM 中的分支通常用于发布和主要的实验性工作。我们观察到，使用 CSCM 的开发人员只为他们自己的个人工作创建分支是不常见的做法，这些工作会定期合并到主线中。相比之下，分支在 git 中是轻量级的。使用 git 的开发人员可以自由地为新功能或错误修复创建分支，对其进行测试，然后在他们有理由相信时将它们合并到发布 / 稳定分支中。git 的一个重要特性是它能够跟踪分支的合并，这是 SVN 所缺乏的。Git 还会创建许多分支，这是去中心化的一个意外副作用（参见 Peril 2，其中讨论了隐式分支）。这种分支与 CSCM 中常见的分支用法有着根本的不同。任何分支的 head 在 SVN 和 git 中都是对该分支进行的最后一次提交。出于挖矿目的，SVN 和 git 标签是相同的。所有 git 存储库都以一个显式分支开头，按照约定，该分支的名称为 master。Git 的 master 与 svn 中的 trunk 类似，但不等同于，参见 Peril 3。

由于 git 会记录有关分支和合并的信息，因此与 SVN 中提交的树状结构相比，存储库的历史记录由提交图表示。在 git 中，一个提交可能有多个父提交（由于合并），也可能是多个提交的父提交（由于分支）。由于没有提交可能是它自己的祖先，因此由提交及其有向父子关系表示的图是有向无环图

(DAG)。DAG 一词在整篇论文中都有使用，在 git 白话中很常见。如果两个 git 提交在 DAG 中具有相同的上级和相同的内容，则它们具有不同存储库中的两个 git 提交具有相同的 SHA-1，这意味着它们的工作副本在创建时是相同的。SHA-1 是内容等效性的高度可靠指标，它允许跟踪从公共来源提取的内容，这允许分别从第三个存储库中提取相同内容的两个存储库相互合并，而无需使用第三个存储库。

git 中的合并会创建一个至少有两个父级的新提交（DAG 中合并的标记）。git 和 SVN 中的合并有两个主要区别：1) n 路合并发生在 git 中，并创建一个具有  $n+1$  父级的合并提交；2) SVN 不明确跟踪合并信息，例如冲突及其解决。当出现合并冲突时，git 会将解决方案写入其提交。此外，每当 SVN 开发人员更新其工作副本时发生的合并永远不会被记录，而

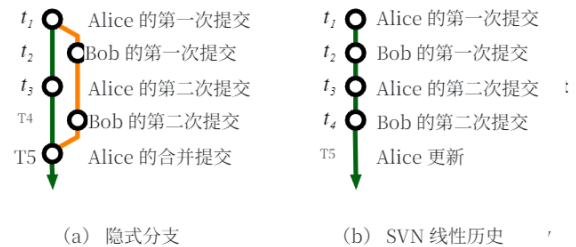


图 3. 相同提交序列的结果。

Git 中的类似作是。我们注意到 Perforce、ClearCase、Accurev 和 SVN 版本 1.5 等工具包括合并跟踪。但是，这些工具创建的 DAG 在语义上是不同的，并且这些工具在 OSS 项目中的普及和使用历史很小（SVN 1.5 仅跟踪代码库切换到 1.5 版本后发生的合并，无法恢复历史合并）。

危险 2：这是“隐含的分支！”（向 Dragons 道歉）

开发人员可以在 git 和 SVN 中显式创建显式分支。在 SVN 中，所有分支都是显式创建的，但 git 并非如此。当两个协作开发人员提交到他们的本地 git 存储库时，他们的存储库会有所不同，即每个存储库都包含另一个存储库中不存在的新提交。当其中一个开发人员从另一个开发人员拉取时，git 会将远程提交序列合并到拉取开发人员的存储库中。这些更改记录为两个分支，从存储库开始分歧的提交（共同的最后一个提交）开始，到 git 创建的合并提交结束。一个分支包含本地开发人员的提交；另一个包含来自远程开发人员的提交。两个开发人员都没有明确创建分支。因此，我们称 pull 可以创建隐式分支的分支。这些分支可能会让那些只熟悉像 SVN 这样的集中式 SCM 的人感到困惑，其中分支总是明确的，指的是替代开发路线。

图 3 (a) 说明了隐式分支。在时间  $t_1$  之前，Alice 和 Bob 的存储库是相同的。在  $t_3$  之后，它们的存储库已分叉：每个存储库都包含彼此未知的提交。Alice 和 Bob 继续在单独的代码线上独立工作，如图所示。在  $t_5$  中，Alice 从 Bob 的存储库中提取更改并解决任何冲突，合并两个存储库，并自动为合并创建提交。在  $t_5$  之前，Bob 的更改对 Alice 不可见；在  $t_5$  之后，Bob 的代码线是 Alice 历史中的一个隐含分支。但是，Bob 的存储库不包含 Alice 的提交。相比之下，图 3 (b) 显示了 Alice 和 Bob 使用 CSCM 进行相同更改时形成的历史记录。因为 Bob 的提交更改了共享的中央仓库，所以 Alice 必须在  $t_3$  更新她的工作副本，并解决出现的任何冲突，然后才能创建它



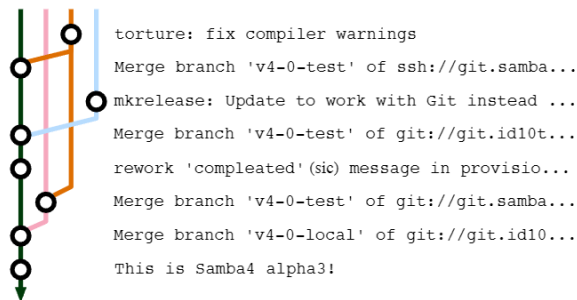


图 4. 导致 Samba 4.0 alpha 3 发布的提交的 git DAG 的子图。

犯；在图 3 (a) 中，Alice 自由地提交到她的本地存储库。同样，Bob 必须在第二次提交之前更新并解决潜在的冲突。在 SVN 中，Alice 和 Bob 分别在不同的开发线上工作的事实已经丢失了。

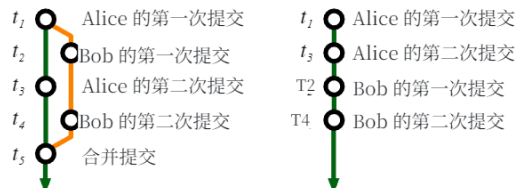
## . Git 日期时间

承诺 2: Git 通过多个仓库的 DAG 中的提交、分支和合并信息，有助于恢复更丰富的项目历史。

由于 git 同时跟踪隐式和显式分支，并在存储库内和存储库之间进行合并，因此它承诺 SVN 不会跟踪数据（SVN 跟踪中央存储库中的分支，但不跟踪合并）。这包括：

- 1) 隐式分支，显示开发人员从其他存储库拉取和推送更改的频率；
- 2) 功能（显式）分支，显示协作活动：直接在开发人员之间拉取的更改，而不是通过中间的“官方”存储库。
- 3) 合并点，包括冲突文件的集合，以及执行合并和解决的人员/时间。
- 4) 从远程存储库拉取，以及“拉取网络”的整体拓扑。
- 5) DAG 和同一项目的不同存储库中的提交集可以确定彼此之间的差异和“距离”。

此数据有多种可能的用途。例如，自发出现的存储库的“拉取网络”是分层的、集中的还是分散的？在自己的分支中开发功能的开发人员之间是否存在明显的协作模式？项目中的状态与开发人员的更改集成到官方存储库的频率或速度之间是否存在关系？



(a) 变基前

(b) 变基后

图 5. 变基示例

风险 3: Git 没有主线，因此必须适当修改分析方法以考虑 DAG。

在 SVN 中，提交的祖先可以被捕获为线性的提交系列，而主干或“mainline”通常以这种方式建模。git 项目开发不是遵循单一的“主线”，而是通过从初始提交到头部的 DAG 中的一组路径。因此，分析方法和存储技术必须处理非线性提交祖先，包括远程祖先。图 4 描述了 Samba 存储库在发布之前发生的分支，显示了这种现象。

在 git 和 SVN 中，开发人员并行工作。在一系列提交中所做的两个功能可以同时进行。但是，由于 SVN 要求开发人员在提交之前进行更新，因此这两个功能的开发历史将变得交错。实际上，在“mainline”上并行开发会导致整个工作被投影到单个日期排序的行中。尽管矿工可以根据需要查看此投影，但不会强制查看此投影，并且必须在使用现有分析方法之前创建投影，或者必须使用其他方法。

危险 4: Git 历史记录是修正主义的：仓库所有者可以重写它。

虽然其他 SCM（如 SVN）也允许重写历史记录，但这很困难且很少这样做。Git 允许用户通过一个称为 rebase 的过程来重写历史记录。用户选择要变基的提交序列；然后，他可以更改提交的顺序、删除提交、将多个提交的编辑压缩为一个提交，或者将多个分支上的一系列提交展平到单个分支上。对提交进行重新排序或展平时，将保留有关提交内容（日期、作者、文件更改）的所有信息，但会修改 DAG 和提交哈希。最常见的变基用例是展平一系列隐式分支。许多项目都有限制使用 rebase 的策略 [18]。在分析从 git 挖掘的数据时，应该了解这些策略。图 5 (b) 显示了什么



图 6. 分支 master 和 feature 之后的 git DAG 彼此合并。

Alice 的 git 存储库看起来就像 5 (a) 中的提交被扁平化（删除分支和合并提交）并重新排序一样。

变基通常用于在将分支拉取到另一个存储库之前“清理”分支。因此，git 仓库（尤其是稳定的官方项目仓库）中的历史记录可能无法反映到达该特定状态的实际情况。使用 rebase 是挖掘卫星存储库的原因之一，这些开发人员使用它来编写新功能，因为这些存储库可能包含完整的、未经修改的开发历史。

危险 5：你不能总是确定提交是在哪个分支上进行的。

尽管提交是在特定 git 存储库的特定分支上显式创建的，但该提交不会记录创建该提交的分支。恢复此信息可能很困难或不可能。图 6 显示了 git DAG 在纵横交错合并后的样子。Alice 将名为 feature 的分支合并到 master，然后将 master 合并到 feature。这些合并共享一个合并提交，如图所示。

在  $t_4$  处交叉合并之前的提交都是两个后续分支的 head 的祖先。提交很少使用写入它们的分支的名称进行注释，并且合并提交的默认提交消息是 merge branch 'master' into feature，这无助于区分其父级。因此，如果存储库是在时间  $t_6$  挖掘的，则通常无法判断合并前的哪些提交是在 master 上进行的（绿线在  $t_4$  之前），哪些是在功能上进行的（橙线在  $t_4$  之前）

风险 6：并不总是能够跟踪合并的来源，甚至确定是否发生了合并。

通常，当开发人员将提交从远程 git 存储库中的某个分支拉取到其本地存储库时，该分支必须合并到当前本地工作分支中。我们想知道该合并的来源，包括



图 7. Alice 的存储库，如果明确告知 git 以避免快进合并 (a)，并且通常在拉取 Bob 的更改后执行快进合并 (b) 时。

术语。这在大多数（但不是全部）条件下是可能的。默认情况下，git 使用以下形式之一为合并提交创建日志消息。括号中的文本可能并不总是显示。

```
Merge branch 'branchname' [into branch_name]
Merge [branch 'branchname' of] remote_repo_url
```

使用此信息以及 DAG 上提交之间的关系，可以查看哪些分支合并在一起以及合并在一起的位置。

无法检测到合并的一种情况是发生快进合并时，如图 7 所示。Alice 向她的 master 进行了两次提交，Bob 将其提取到他的存储库中，然后进行了两次提交。在  $t_4$  之后，如果 Alice 要提取 Bob 的更改，人们会期望她的历史如图 7 (a) 所示。但是，自从 Bob 从存储库中提取后，她的存储库中没有任何变化，因此实际上不需要进行合并。除非 git 被明确告知不要这样做，否则它会按顺序添加 Bob 的提交，并将 Alice 的 HEAD “快进”到从 Bob 拉取的最后一个提交。在此方案中，不会创建合并提交。

提交消息中不存在合并源的情况如下：

- 1) 如果两个分支的合并导致快进合并，则不会创建合并提交，因此日志中将不存在包含该字符串的日志消息。
- 2) 如果在合并期间存在冲突，开发人员将解决冲突，并使用可能不包含默认文本的日志消息提交其解决方案。
- 3) 如果开发人员对一系列包含分支和合并的提交进行变基，则结果可能会被“扁平化”，并且不包括合并提交。
- 4) 如果开发人员将合并提交的提交消息修改为默认合并消息以外的内容（这不太可能，但有可能）。

图 8 说明了这种危险。在  $t_3$ ，Bob 在 Carol 中提取时创建了一个合并提交；其提交消息指定 Carol 存储库的 URL。假设 Alice 在从 Bob 中提取时覆盖了  $t_5$  处的默认合并文本。在 Alice 的 DAG 中，知道哪个分支是 Bob 的，哪个是 Carol 的，这并非易事。挖掘 Alice 的存储库，人们会错过 Alice 从 Bob 中提取的事实，并且



图 8. 从 Bob 的存储库中提取后的 Alice's 存储库。日志中没有明确的信息记录 Alice 从 Bob 那里拉取，人们可以错误地推断 Alice 直接从 Carol 那里拉取。

可能会错误地得出 Alice 从 Carol 那里抽离的结论。任何基于存储库之间拉取信息的分析都应该对这些问题敏感。

在上述前两种情况下，没有证据表明发生了合并。由于无法确定由于这些限制而错过了多少次合并，因此矿工在执行任何依赖于分支和合并的分析时必须小心。在最后两个中，仍然有证据，因为有一个具有两个父级的 commit，但是源信息可能会丢失。由于在这些情况下合并是已知的，因此可以衡量信息损失我们发现，对于 30 个 git 项目的样本，使用 heurmetrics 在 98% 的情况下有效。分析和结果的详细信息在第 5 节中介绍。

承诺 3: Git 在私有日志中记录更正风险 3-6 所需的信息。

当 Bob 克隆 Alice 和 Ram 的存储库时，他对这些存储库的历史记录的看法被掩盖了。他无法确定 Alice 和 Ram 何时以及从谁那里拉取，他们何时以及如何变基，等等。但是，此信息不一定会丢失，即使它不是很容易访问的。Git 将有关快进合并、变基和拉取的信息存储在 logs 目录中。事实上，可以在此处找到有关开发人员工作流程的更多信息 — 每次签出都会被记录下来，因此研究人员可以观察开发人员何时在分支之间切换。

这有望挖掘数据，其中包括有关开发人员工作流程和项目历史记录的大量信息，但仅限于特定条件。矿工必须有权访问她希望挖掘其日志的每个开发人员的私有存储库。因此，很有可能编写利用此信息的工具，供组织用于其内部项目。

在某些情况下，研究人员可能很幸运地可以访问或复制一些他们希望挖掘的项目私有开发人员存储库。当这个机会出现时，研究人员必须意识到，所获得的信息比

定期存储库克隆。

存在一个警告。Git 有一个清理命令 gc。正常执行 gc 不会干扰日志文件，但它有一个激进模式。如果开发人员一直在运行 gc --aggressive，则某些日志条目可能已被删除。

风险 7: 可访问的数据只能包含成功选择的提交。

由于其他原因，历史记录可能会丢失或修改。在一个工作流程中，开发人员创建一个分支，其中包含要由其他开发人员提取和审查的更改。一旦进行审核，如果更改未被接受，则 Branch 可能会被销毁。或者，开发人员可以通过在包含之前变基或添加提交来修改分支。这个过程类似于在 SVN 项目中提交补丁以供审核：该补丁可能会被拒绝或最终提交（在可能的修改之后）。就像在基于 SVN 的项目中恢复补丁和审查过程很困难一样 [19]、[20]，提交的审查历史不会直接反映在 git 中。最初，我们原本认为我们会在 git 历史中看到更多的审查过程。我们发现，在将邮件列表与 signed-off-by、reviewed-by 和 test-by 字段一起使用时，它们取得了一些成功。

承诺 4: signed-off-by 和其他属性创建“书面记录”。

作为对 SCO 提出的 IP 侵权指控的回应，Linus Torvalds 通过在提交消息的末尾添加一行，例如 signedoffby John Doe jdoe@foobar.com，为人们添加了一个工具，供人们“签署”提交。还有其他属性：ackedby、Cc、reportedby、reviewedby 和 testedby。这些属性使用 -s 标志显式添加到提交中，并使用易于解析的标准格式在提交日志消息中附加一行。提交在存储库之间移动并经过审查和测试时，可能会附加多个字段。

此信息可用于（例如）确定社区中的某些角色，例如审阅者或测试人员。它还可用于评估项目中的专业知识或重建社区的组织。我们在第 5 节中展示了这些数据在确定专业知识和可视化 linux 社会组织层次结构中的一种用途。

到目前为止，我们观察到 Linux 内核之外很少有项目使用这些工具，但我们预计，随着具有更严格执行策略的项目采用 git，它们将利用这一额外的功能来记录有关提交历史的信息。



承诺 5: Git 显式记录不属于核心开发人员集的贡献者的作者信息。

在使用集中式 SCM 的传统 OSS 项目中，有一组显式的开发人员对源代码存储库具有写入权限。存储库日志指示哪些开发人员进行了每项更改。然而，OSS 项目严重依赖志愿服务 [12]，并且许多社区成员以补丁的形式做出贡献，通常提交给开发邮件列表。

如果补丁被接受，则核心开发人员会将其提交到存储库，并且日志指示更改归属于该开发人员。理想情况下，我们希望能够将源代码更改归功于做出更改的人。不幸的是，检测提交的补丁的应用程序是困难的 [19]。在某些项目（如 Apache）中，开发人员应用提交的补丁时，通常会在提交消息中包含归属信息。但是，所收集归因数据的准确性取决于项目、开发人员是否遵循惯例以及日志消息格式的标准化。Git 存储库具有更准确的作者信息，因为它可以显式地自动包含有关更改作者的信息。

基于 git 的项目的贡献者可以通过两种方式提交更改。任何具有读取访问权限的人都可以克隆公共 git 存储库并提交到他们自己的副本。任何贡献者都可以要求项目开发人员将更改从自己的存储库提取到主项目存储库中。此外，git 还提供了一种工具，可以将提交或提交序列转换为特殊格式的补丁，该补丁可以通过电子邮件发送给项目开发人员并应用于主项目存储库。即使在补丁的情况下，作者信息也会在提交到“官方”存储库时自动提取和保留。这种记录保留是 git 提交具有 committer 字段（用于标识谁将更改提交到存储库）和 author 字段（用于标识进行更改的人员）的原因之一。作者信息在存储库之间传输时与提交相关联。我们在第 5 节中对归因问题进行了实证研究。

## · 挖掘 Git 金矿脉

承诺 6: 在 git 中，所有元数据，尤其是历史元数据，都是本地的。

当开发人员基于预先存在的存储库创建本地存储库时，远程存储库中的所有信息都将传输到本地存储库。过去

|                                                             |                                                             |
|-------------------------------------------------------------|-------------------------------------------------------------|
| <pre>#include &lt;math.h&gt; int add(int a) {   ... }</pre> | <pre>#include &lt;math.h&gt; int sub(int a) {   ... }</pre> |
| <pre>int sub(int a) {   ... }</pre>                         | <pre>int add(int a) {   ... }</pre>                         |
| <pre>void main() { ... } (a) original by Alice</pre>        | <pre>void main() { ... } (b) modified by Bob</pre>          |

图 9. 文件的两个版本

分析需要分析仓库中每个文件的每个版本时，我们依靠第三方工具，如 CVSSuck [21] 或 svn::mirror [22] 来创建本地镜像，以便我们可以快速签出文件，而不会给官方公共仓库带来负担。不幸的是，我们发现这些工具容易出错，有时还会有损。在某些情况下，我们需要联系项目维护者以请求其源代码存储库的副本。由于有关 git 存储库的所有信息都存储在本地，因此无需发出请求或使用第三方工具。所有分布式 SCM 都有这种优势，而且并非 git 所独有。当然，元数据会有所不同，因为同一项目中的存储库会有所不同。

承诺 7: Git 跟踪内容，因此它可以在移动或复制行时跟踪行的历史记录。

Git 跟踪文件内容的历史记录。这意味着它能够自动判断文件是否被重命名，因为内容保持不变，并且历史记录跟随内容。在 SVN 中，开发人员必须显式重命名文件才能维护文件的历史记录。此外，git 能够跟踪文件内部和文件之间的大量文本。考虑图 9 中文件的两个版本。Alice 编写了第一个版本。Bob 在将函数 sub 移动到 add 上方而不更改其内容时创建了第二个版本。

如果运行 svn blame 来显示谁引入了每一行，则 Bob 将被指示为函数 sub 中每一行的作者。但是，git blame -C -M（其中 -C 查找代码副本，-M 查找代码移动）表示 Alice 已经编写了文件的每一行。只要移动的行数超过某个阈值（我们观察到阈值约为 4 或 5），Git 还会跟踪在文件之间移动的文本和被复制多次的文本。当然，如果 Bob 要修改实现中的一行，哪怕是一点点，git 都会将该行的作者身份分配给 Bob。Git 的源站分析不如最近引入的研究技术准确 [23]，但它比 SVN 有了巨大的改进。



承诺 8: Git 比集中式存储库更快, 并且通常使用更少的空间。

Git 旨在处理 Linux 大小的代码库并管理大量贡献。由于元数据位于本地计算机上, 因此访问日志或 diff 在本地执行, 没有网络延迟。此外, git 存储文件的完整 (压缩) 版本, 而不是一系列 diff。这意味着签出文件是一个 constant time 作, 与历史记录无关。

我们将整个 Eclipse CVS 存储库转换为 git 格式, 并注意到日志访问和签出必须更快。使用 git 以任意 (非连续) 顺序签出提交, 每次提交需要 1-2 分钟, 而使用 CVS 需要 7-25 分钟。使用 git 签出连续提交每次提交需要 0.5-0.8 秒, 而使用 CVS 在我们的服务器上签出存储库和工作副本需要 1-3 分钟。这使得一些以前很痛苦的表单分析变得可行。

尽管设计选择存储整个修订版, 但 Git 的整体压缩策略使用的空间比 CVS 或 SVN 少得多。整个 Mozilla 存储库存储在 SVN 中时, 大小为 12 GB。然而, 当仓库被导入到 git 中时, 它缩小到 420 MB [24]。我们发现 Eclipse CVS 存储库使用 8.6 GB, 而 git 使用 3.3 GB。python 社区发现, 从 SVN 转换为 git 可将磁盘使用量从 1.3 GB 减少到 150 MB [25]。

承诺 9: 大多数 SCM, 如 CVS、SVN、Perforce 和 Mercurial (Hg), 都可以转换为 git, 并且分支、合并和标签的历史记录保持不变。

由于 git 在过去几年中的流行, 围绕它的开发社区开发了许多工具, 用于从最流行的 SCM3 迁移到 git。其中许多工具仍在积极开发中, 并且是官方 git 发行版的一部分。例如, 有许多 git svn 命令充当 SVN 存储库和本地 git 存储库之间的双向网关。我们已经成功地将整个 Eclipse CVS 存储库转换为带有标签和分支的 git。我们还将 NetBeans 和 Mozilla Central 的 Mercurial (Hg) 存储库以及许多其他 SVN 存储库导入到 git 中。通过将几乎所有存储库转换为 git 格式, 我们已经能够获得 git 的一些好处, 例如更好的源检测、性能和所有信息的本地副本。此外, 我们只需要为一种格式编写挖掘和分析工具, 而不是多种格式。

3. 查看 <http://git.or.cz/gitwiki/InterfacesFrontendsAndTools> 以获取完整列表。

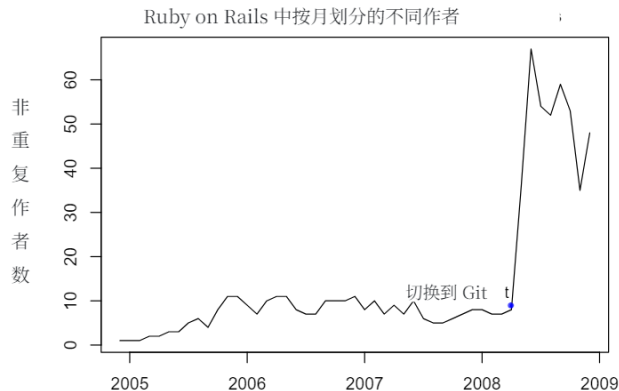


图 10. 存储库日志报告的 Ruby on Rails 的每月作者数。蓝点表示 Rails 迁移到 git 的日期。

## 分析

Promise 5 声称 git 记录的信息和启用的工作流允许更准确的作者分析。为了检验这一说法, 我们从项目中提取了数据, 该项目在可衡量的时间段内同时使用了 SVN 和 git, 并比较了作者信息。图 10 显示了按月向 Ruby on Rails 项目存储库贡献更改的不同作者的数量。图表上的点表示项目迁移到 git 的时间。虽然迁移到 git 可能会导致贡献者数量急剧增加, 但这种增加似乎更有可能是由于 git 在跟踪作者身份方面的卓越功能。

根据 Peril 6, git 中的日志消息会记录合并的来源。虽然可以从日志中启发式地恢复合并信息, 但它可能不完整。我们通过对 <http://git.or.cz> 中列出的 30 个使用 git 的 OSS 项目中挖掘数据, 实证检查了这些启发式方法的召回率。

or.cz/gitwiki/GitProjects 的大小从 9 个作者 (disco) 到超过 1000 个 (wine) 不等, 包含多达 139,187 个提交

(samba)。如果项目从另一个 SCM 迁移, 并且 git 导入了历史记录, 我们只检查了在过渡到 git 后创建的合并。总共有 2,971 次合并 (即具有多个父级的提交)。我们忽略了无法检测到的快进合并, 或由于变基而扁平化的合并。我们的启发式方法能够在 2,909 个案例中检测到合并的来源, 召回率为 97.9%。对于大多数用途来说, 这似乎在可接受的范围内。但是, 研究人员在根据合并来源解释和报告结果时, 应执行类似的分析。

我们假设 git 的使用可能会影响工作流模式, 并且开发人员在项目从集中式切换到 git 后会进行更小、更频繁的提交

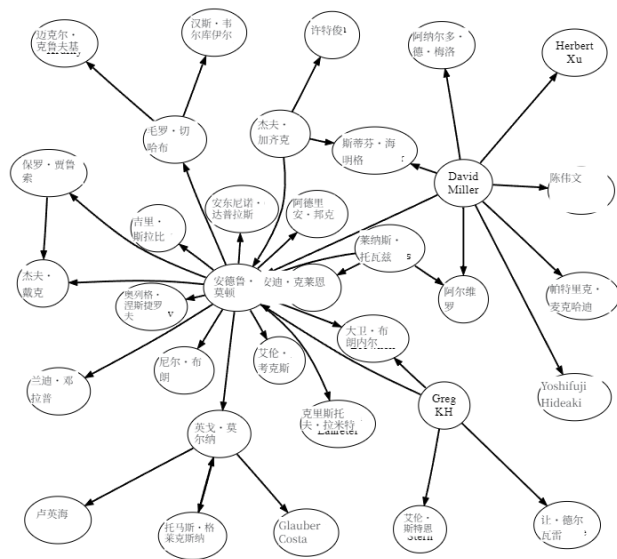


图 11.signed-off-by 网络。从 a 到 b 的边表示 a 在 b 之后立即签署了提交。

单片机。因此，我们对许多项目在迁移到 git 之前和之后的提交大小进行了分析，以添加和删除的 LOC 来衡量。尽管在统计上显著，但差异的大小并不有趣（每次提交仅少 2 行）。

为了评估 Promise 4，我们研究了日志消息中包含的归因数据，例如 signed-off-by。作为对 Linux 内核中开发过程和 workflows 的更广泛研究的一部分，我们发现（例如）大多数与非驱动程序相关的网络和 SPARC 架构相关的提交都是由 David Miller 签署的（分别为 69% 和 72%，这两个领域占他总签署的 92%）。在 Miller 修改的所有文件中，87% 与网络有关，10% 与 SPARC 有关。git 数据分析的这一结果证实了民间传说，即他既是这些领域的守门人，也是专家。

signed-off-by 网络还允许我们调查 signing-off 和创作贡献之间的关系。Torvalds 表示，他目前在 Linux 中的角色是集成商，而不是开发人员。从数量上讲，我们发现这种说法是正确的：他编写了 632 次提交，执行了 4,317 次合并，并签署了 25,456 次提交。整个社区中签字数量与创作贡献数量之间的非参数相关性是中等的 ( $r = .65$ ,  $p \ll .001$ )。这表明，深度参与签字的开发人员（即层次结构中的较高级别）通常自己做的开发工作较少，并且社区中存在不同的角色。

对开发人员在“拉取网络”中的位置和文件位置的全面研究有助于确定社区成员所承担的负载、他们的专业领域、状态或信任级别以及角色（审阅者与提交者）。我们根据开发人员对提交进行签核的顺序重新创建了 signed-off-by 网络。图 11 描述了该网络的一部分，包括层次结构的关键成员和它们之间的最高加权边。

用于执行这些分析的数据仅从公共 git 存储库中提取，并使用我们编写的工具存储在 PostgreSQL 数据库中。这些工具可供 [git:// github.com/cabird/git\\_mining\\_tools](https://github.com/cabird/git_mining_tools) 的其他研究人员免费使用。

## 总结

DSCM 承诺提供新的有用数据，以帮助我们更好地了解软件流程。这些包括准确的作者信息；识别审阅者、提交者和作者等角色的能力；同一项目中开发人员之间的存储库内容差异；和合并跟踪。与所有数据一样，必须谨慎处理这些数据。我们概述了在挖掘 DSCM 数据时可能遇到的一些主要陷阱。值得注意的是，SVN 和 DSCM 之间提交和分支的语义不同；一些元数据，比如 DAG 可以被开发人员修改，而且通常无法知道提交发生在哪个分支上（这种危险让我们花费了几天的工作）。

我们计划使用这些数据来解决以下问题：1) 从集中式 SCM 切换到 DSCM 时，项目中的开发流程或通信网络是否会发生变化？2) 使用 DSCM 是否会导致更专注的开发，但项目意识更差？3) 开发人员组是否协同工作，在一段时间内独立于“官方”存储库？我们希望这篇论文能增强其他研究人员收集、分析和解释这些数据以回答研究问题的能力。

## 引用

- [1] T. Zimmermann 和 P. Weißgerber, “预处理 CVS 数据以进行细粒度分析”，载于《采矿软件存储库国际研讨会论文集》，2004 年。
- [2] M. Fischer, M. Pinzger, and H. Gall, “Populating a release history database from version control and bug tracking systems,” in ICSM '03: Proceedings of the International Conference on Software Maintenance. 美国华盛顿特区: IEEE 计算机学会, 2003 年, 第 23 页。
- [3] D. Cubranic, G. Murphy, J. Singer 和 K. Booth, “Hipikat: 软件开发的项目内存”，软件工程, IEEE Transactions on, 第 31 卷, 第 6 期, 第 446-465 页, 2005 年。

- [4] T. Zimmerman、A. Zeller、P. Weissgerber 和 S. Diehl, “挖掘版本历史记录以指导软件更改”, 软件工程, IEEE Transactions on, 第 31 卷, 第 6 期, 第 429-445 页, 2005 年。
- [5] B. Dagenais 和 M. P. Robillard, “为框架演变推荐适应性变化”, ICSE, 2008 年, 第 481-490 页。
- [6] B. Fluri、H. Gall 和 M. Pinzger, “变更耦合的细粒度分析”, 源代码分析和作, 2005 年。第五届 IEEE 国际研讨会, 第 66-74 页, 2005 年。
- [7] H. Gall 和 M. Lanza, “软件进化: 分析和可视化”, 2006 年软件工程国际会议论文集, 第 1055-1056 页, 2006 年。
- [8] M. Pinzger、H. Gall、M. Fischer 和 M. Lanza, “可视化多个进化指标”, 2005 年 ACM 软件可视化研讨会论文集, 第 67-75 页, 2005 年。
- [9] S. Kim、T. Zimmermann 和 K. Pan, “自动识别错误引入更改”, 第 21 届 IEEE 自动化软件工程国际会议 (ASE'06) 论文集 - 第 00 卷, 第 81-90 页, 2006 年。
- [10] S. Kim、T. Zimmermann、E. Whitehead Jr 和 A. Zeller, “从缓存的历史记录中预测故障”, 第 29 届软件工程国际会议论文集, 第 489-498 页, 2007 年。
- [11] S. Kim 和 M. D. Ernst, “通过分析软件历史记录来确定警告的优先级”, MSR 2007: 采矿软件存储库国际研讨会, 美国明尼苏达州明尼阿波利斯, 2007 年 5 月 19 日至 20 日。
- [12] C. Bird, A. Gourley, P. Devanbu, A. Swaminathan, and G. Hsu, “开放边界? 开源项目中的移民”, MSR '07: 第四届采矿软件存储库国际研讨会论文集。美国华盛顿特区: IEEE 计算机学会, 2007 年, 第 6 页。
- [13] C. Bird, D. Pattison, R. D'Souza, V. Filkov, and P. Devanbu, “开源项目中的潜在社会结构”, SIGSOFT '08/FSE-16: 第 16 届 ACM SIGSOFT 软件工程基础国际研讨会论文集。美国纽约州纽约: ACM, 2008 年, 第 24-35 页。
- [14] A. Mockus、J. D. Herbsleb 和 R. T. Fielding, “开源软件开发的两个案例研究: Apache 和 mozilla”, ACM 软件工程和科学方法汇刊, 第 11 卷, 第 3 期, 第 309-346 页, 2002 年 7 月。
- [15] B. Massey 和 K. Packard, “反刍: 使用 GIT 进行 F/LOSS 数据收集”, 第 1 届软件开发公共数据国际研讨会, 2006 年。
- [16] G. KroahHartman, “Linux 内核开发”, Linux 研讨会, 2007 年, 第 239-244 页。
- [17] J. Corbet, “补丁如何进入主线”, Linux 周报, 2009 年 2 月 10 日。[在线]。可用: <http://lwn.net/Articles/318699/>
- [18] “Git 管理。” [在线]。可用: <http://kerneltrap.org/Linux/Git管理>
- [19] C. Bird、A. Gourley 和 P. Devanbu, “检测 OSS 项目中的补丁提交和验收”, 第 4 届采矿软件存储库国际研讨会论文集, 2007 年。
- [20] P. Rigby、D. German 和 M.-A. Storey, “开源软件同行评审实践: Apache 服务器的案例研究”, 载于国际软件工程会议论文集, 2008 年。
- [21] “CVS 很烂。” [在线]。可用: <http://cvs.m17n.org/~akr/cvssuck/>
- [22] “SVN 镜像。” [在线]。可用: <http://search.cpan.org/dist/SVN-Mirror/>
- [23] G. Canfora、L. Cerulo 和 M. D. Penta, “从版本存储库中识别更改的源代码行”, 第 4 届采矿软件存储库国际研讨会论文集, 2007 年。
- [24] “Git SVN 比较。” [在线]。可用: <http://git.or.cz/gitwiki/GitSvnComparsion>
- [25] “DVCS 后续行动: 管理 Python 存储库。” [在线]。可用: <http://ldn.linuxfoundation.org/blog-entry/dvcs-follow-what-about-managing-python-repository>